Práctica Socket

Integrantes: Alonso Winter Constanza
Fontes Samuel

1. Introducción

El presente trabajo tiene como objetivo implementar una aplicación en Java utilizando sockets, que permita la comunicación entre un cliente y un servidor mediante un sistema de chat interactivo.

Se busca aplicar los conceptos teóricos vistos en clase, desarrollando una arquitectura cliente-servidor donde ambos programas se ejecutan de manera independiente y se comunican a través de una red local.

2. Tecnologías utilizadas

- Lenguaje: Java
- IDE: Apache NetBeans
- Comunicación: Sockets TCP (`java.net.Socket`, `ServerSocket`)
- Entrada/Salida: `BufferedReader`, `PrintWriter`, `Scanner`

---

3. Estructura del proyecto

El proyecto está compuesto por dos clases principales:

- `Servidor.java`
- `Cliente.java`

Cada clase posee su propio método `main`, lo que permite ejecutarlas de forma independiente.

---

4. Funcionamiento del sistema

Servidor

El servidor:

- Se inicia en el puerto 5000.
- Permanece en espera de conexiones.
- Acepta un cliente.
- Recibe mensajes enviados por el cliente.
- Procesa los mensajes según comandos definidos.
- Envía respuestas al cliente.
- Muestra en consola un log de toda la comunicación.

Cliente

El cliente:

- Se conecta al servidor en `localhost:5000`.
- Permite al usuario ingresar mensajes desde teclado.
- Envía los mensajes al servidor.
- Recibe y muestra la respuesta del servidor.
- Finaliza la conexión cuando se envía el comando `SALIR`.

---

5. Interacción cliente-servidor

El sistema funciona como un chat:

Cliente → envía mensaje
Servidor → procesa y responde
Cliente → muestra respuesta

---

6. Comandos implementados

RESOLVE

Permite resolver expresiones matemáticas.

Ejemplo:

```
Output
Socket (run)   Socket (run) #2
run:
╔═══════════════════════════════════════╗
║       CLIENTE DE CHAT - SOCKET        ╕
╚═══════════════════════════════════════╝
[CLIENTE] Conectando a localhost:5000...
[CLIENTE] ✔ Conectado al servidor!

╔═══════════════════════════════════════════════════╗
║ COMANDOS DISPONIBLES                              ╕
│                                                   │
│  RESOLVE "expresion"  -> Calcula una expresión mat │
│    Ejemplo: RESOLVE "45*23/54+234"                │
│    Ejemplo: RESOLVE "(100+50)*2"                  │
│    Ejemplo: RESOLVE "2^10"                        │
│  MAYUS "texto"   -> Convierte un texto a mayúsculas │
│    Ejemplo: MAYUS "hola mundo"                    │
│                                                   │
│  AYUDA                -> Ver comandos del servidor │
│  SALIR                -> Cerrar la conexión       │
│                                                   │
│  También podés escribir cualquier mensaje de chat. │
╚═══════════════════════════════════════════════════╝

Vos >> RESOLVE "6*8"
Servidor >> Resultado de [6*8] = 48

Vos >> |
```

MAYUS

Convierte un texto a mayúsculas.

Ejemplo:

```
Output
Socket (run)   Socket (run) #2
run:
╔═══════════════════════════════════╗
║       CLIENTE DE CHAT - SOCKET    ╕
╚═══════════════════════════════════╝
[CLIENTE] Conectando a localhost:5000...
[CLIENTE] ✔ Conectado al servidor!

┌──────────────────────────────────────────────┐
│ COMANDOS DISPONIBLES                         │
│                                              │
│  RESOLVE "expresion"  -> Calcula una expresión mat │
│    Ejemplo: RESOLVE "45*23/54+234"           │
│    Ejemplo: RESOLVE "(100+50)*2"             │
│    Ejemplo: RESOLVE "2^10"                   │
│  MAYUS "texto"   -> Convierte un texto a mayúsculas │
│    Ejemplo: MAYUS "hola mundo"               │
│                                              │
│  AYUDA                -> Ver comandos del servidor │
│  SALIR                -> Cerrar la conexión  │
│                                              │
│  También podés escribir cualquier mensaje de chat. │
└──────────────────────────────────────────────┘

Vos >> MAYUS "hola profe"
Servidor >> HOLA PROFE

Vos >> |
```

AYUDA

Muestra los comandos disponibles.

Ejemplo:

```
Output
Socket (run)   Socket (run) #2

[CLIENTE] Conectando a localhost:5000...
[CLIENTE] ✔ Conectado al servidor!

 COMANDOS DISPONIBLES

  RESOLVE "expresion"  -> Calcula una expresión mat
    Ejemplo: RESOLVE "45*23/54+234"
    Ejemplo: RESOLVE "(100+50)*2"
    Ejemplo: RESOLVE "2^10"
  MAYUS "texto"   -> Convierte un texto a mayúsculas
    Ejemplo: MAYUS "hola mundo"

  AYUDA                -> Ver comandos del servidor
  SALIR                -> Cerrar la conexión

  También podés escribir cualquier mensaje de chat.

Vos >> AYUDA
Servidor >> Comandos: RESOLVE "expresion" | MAYUS "texto" | SALIR | AYUDA
```

SALIR

Finaliza la conexión entre cliente y servidor.

Ejemplo:

```
Vos >> SALIR
Servidor >> Hasta luego! Conexión cerrada por el servidor.

[CLIENTE] Conexión finalizada. ¡Hasta luego!
BUILD SUCCESSFUL (total time: 39 seconds)
```

7. Log del servidor

El servidor muestra en consola todos los mensajes recibidos:

Ejemplo:

```
Output
Socket (run) ×   Socket (run) #2 ×
run:
╔═══════════════════════════════╗
║       SERVIDOR DE CHAT - SOCKET       ╒
╚═══════════════════════════════╝
[SERVIDOR] Iniciando en puerto 5000...
[SERVIDOR] Esperando conexión de un cliente...

[SERVIDOR] ✔ Cliente conectado desde: 127.0.0.1
[SERVIDOR] --- Inicio de sesión ---

[LOG] Cliente dice: AYUDA
[LOG] Servidor responde: Comandos: RESOLVE "expresion" | MAYUS "texto" | SALIR | AYUDA

[LOG] Cliente dice: RESOLVE "6*8"
[LOG] Servidor responde: Resultado de [6*8] = 48

[LOG] Cliente dice: MAYUS "coni"
[LOG] Servidor responde: CONI

[LOG] Cliente dice: SALIR
[SERVIDOR] El cliente solicitó desconectarse.

[SERVIDOR] --- Sesión cerrada ---
[SERVIDOR] Esperando nueva conexión...
```

Esto permite monitorear la actividad del sistema.